

Python Essentials: Weeks 2–4

Logic, Functions, and Files

CMPT 211

Concordia University of Edmonton

Winter 2026

Structured Programming

We don't just "write code." We build structures.

Every program you write in this course is built from three blocks:

- ➊ **Selection (Week 2):** Making decisions (`if`).
- ➋ **Iteration (Week 2):** Repeating tasks (loops).
- ➌ **Modularization (Week 3):** Grouping tasks (functions).

1. Selection (The if Statement)

Concept: The program chooses a path based on data.

```
grade = 85

# The Indentation (spaces) is REQUIRED
if grade >= 50:
    print("Pass")
    print("Good job!")

else:
    print("Fail")
    print("Try again.")
```

2. Iteration (The for Loop)

Concept: Running the same code for every item in a list.

```
# A list of names
students = ["Alice", "Bob", "Charlie"]

# 's' is a temporary variable for the current item
for s in students:
    print(f"Hello, {s}")

print("Done greeting everyone.")
```

3. Modularization (Functions)

Concept: Giving a name to a block of code so we can reuse it.

- **def:** Defines the function.
- **return:** Sends the answer back.

```
def calculate_tax(price):  
    """Adds 5% tax to a price."""  
    total = price * 1.05  
    return total  
  
# Using the function  
my_coffee = calculate_tax(4.00)  
print(my_coffee)    # Output: 4.2
```

Why return matters

Crucial Concept: `print()` shows text on the screen. `return` gives the data back to the program.

```
def bad_math(a, b):  
    print(a + b)    # Shows '5', but returns Nothing  
  
def good_math(a, b):  
    return a + b    # Returns 5  
  
result = good_math(2, 3)  
print(result * 10)  # We can use the result! (50)
```

4. Stability (Handling Errors)

Concept: Programs crash. We use try/except to catch them.

```
user_input = "five"

try:
    # This line will crash
    number = int(user_input)
    print("Success!")

except ValueError:
    # This runs if it crashes
    print("That is not a number!")
```

5. Files (Reading Data)

Concept: Reading a file safely using with open.

```
filename = "data.txt"

try:
    with open(filename, "r") as f:
        content = f.read()
        print(content)

except FileNotFoundError:
    print("Error: Could not find the file.")
```


Putting it all together

A complete script using Loop, Function, and Error Handling.

```
def process_grades(grade_list):  
    for g in grade_list:  
        try:  
            if g >= 50:  
                print(f"Pass: {g}")  
            else:  
                print(f"Fail: {g}")  
        except TypeError:  
            print("Invalid Data Found")  
  
# Run it  
data = [80, 45, "Error", 90]  
process_grades(data)
```

Advanced Functions: Recursion

Concept: A function that calls *itself*.

To stop it from running forever, you need a "**Base Case**" (The Stop Button).

```
def countdown(n):  
    # 1. Base Case (When to stop)  
    if n <= 0:  
        print("Blastoff!")  
  
    # 2. Recursive Step (Call itself with a smaller number)  
    else:  
        print(n)  
        countdown(n - 1)  
  
# Run it  
countdown(3)
```

Recursion Example: Factorial

The Math: $5! = 5 \times 4 \times 3 \times 2 \times 1$

In code, we say: "The factorial of 5 is $5 \times$ the factorial of 4."

```
def factorial(n):  
    # 1. Base Case (The Stop Button)  
    # 0! is defined as 1  
    if n == 0:  
        return 1  
  
    # 2. Recursive Step (The Loop)  
    else:  
        # "Me times the factorial of the number below me"  
        recurse = factorial(n - 1)  
        return n * recurse  
  
# Testing it
```